Hochschule Konstanz
Fakultät Informatik

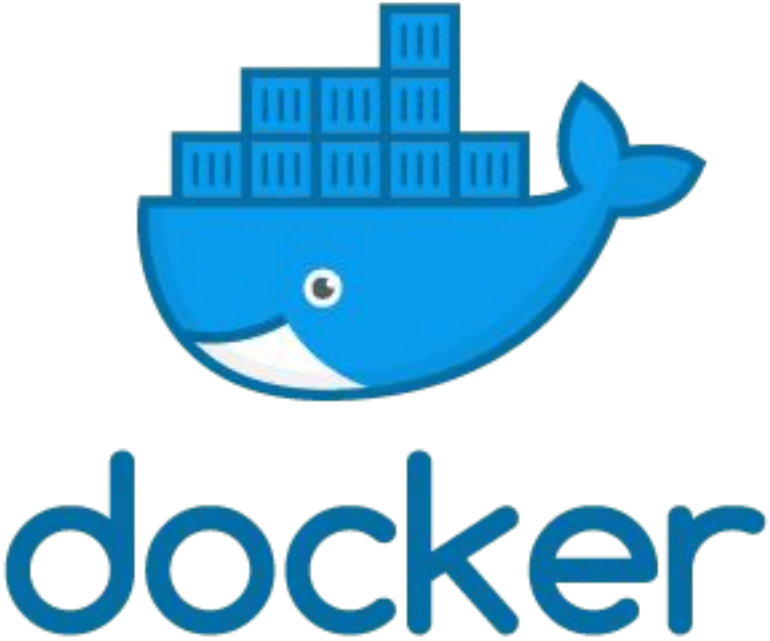
Software Architecture

Docker Container Management

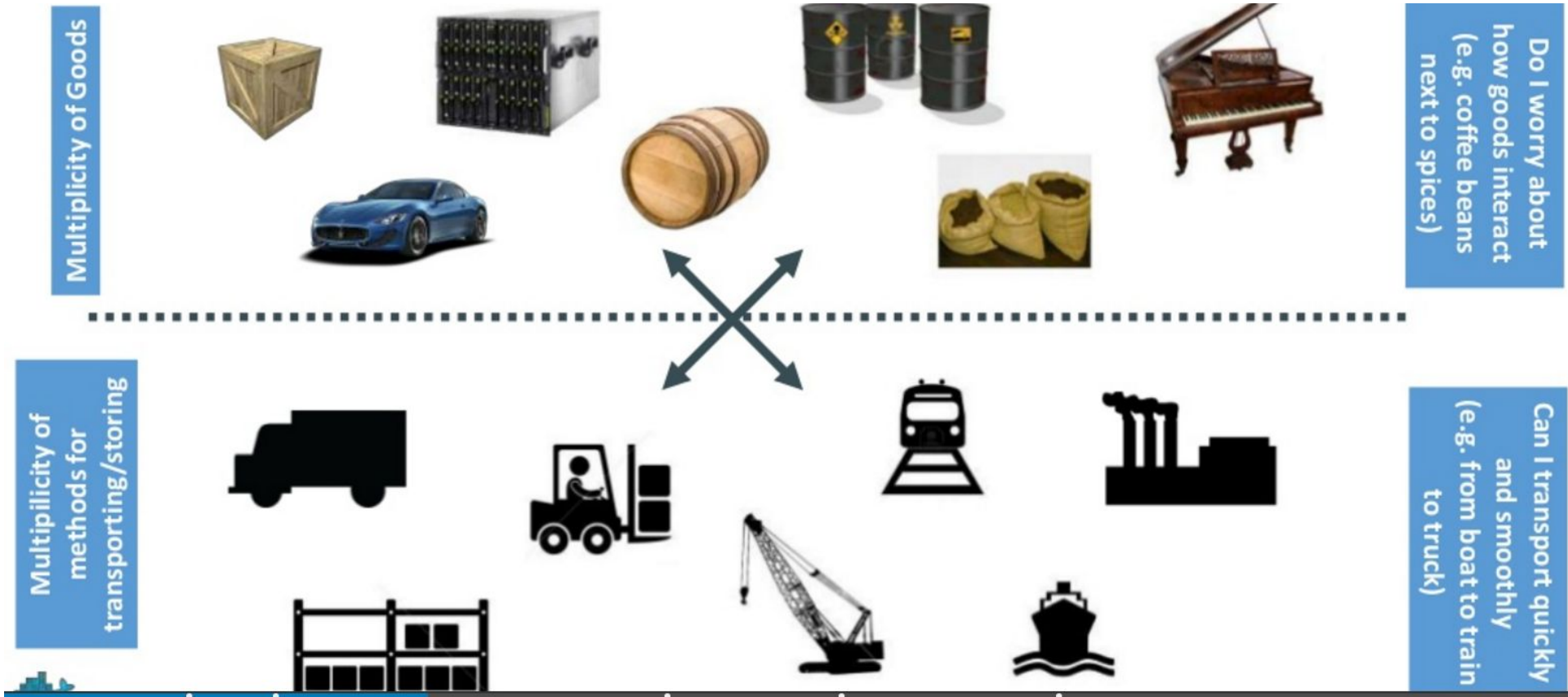
Prof. Dr. Marko Boger

Docker

Docker is an engine to run containers. So, what are containers?



An Analogy:
Transportation in the 60's



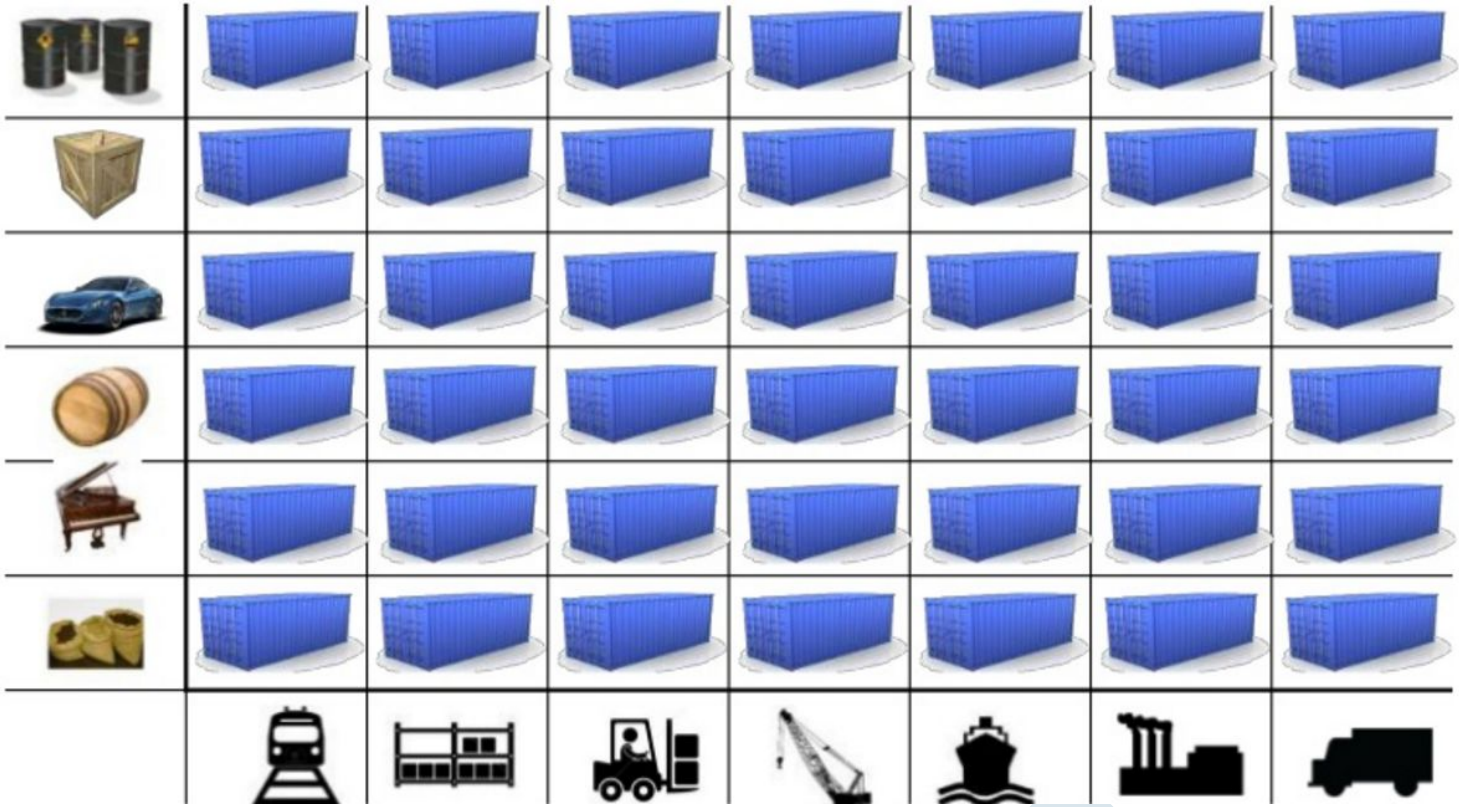
Hochschule Konstanz
Fakultät Informatik

Individual Solutions for every Platform and Good

	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?
	?	?	?	?	?	?	?

The Solution for Transport: Shipping Containers

One Solution for all Platforms and all Goods



Today, shipping is done with Containers

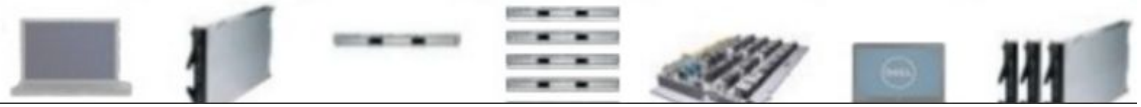
- 90% of all cargo now shipped in a standard container
- Order of magnitude reduction in cost and time to load and unload ships
- Massive reduction in losses due to theft or damage
- Huge reduction in freight cost as percent of final goods (from >25% to <3%)

→ massive globalizations

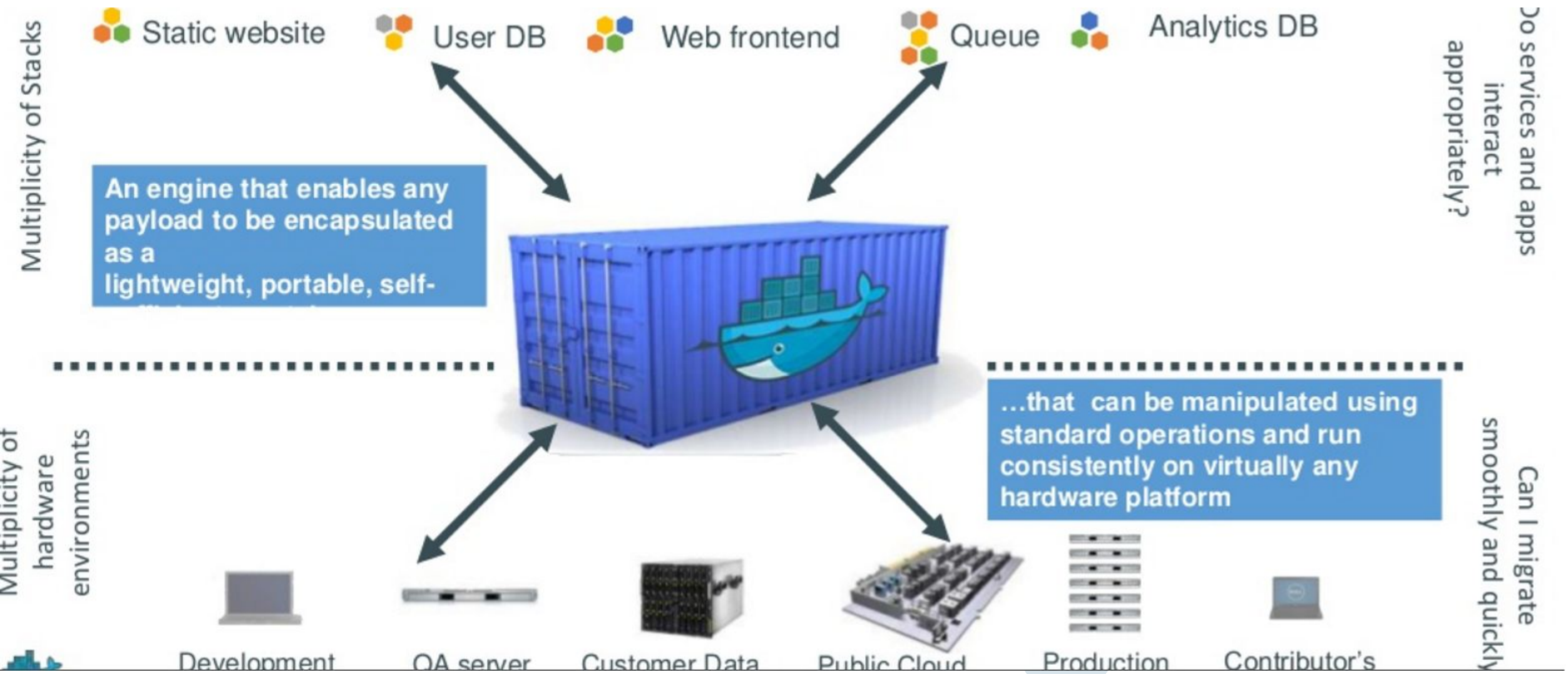
- 5000 ships deliver 200M containers per year

In Software we have a similar Problem

	Development VM	QA Server	Single Prod Server	Onsite Cluster	Public Cloud	Contributor's laptop	Customer Servers
Static website	?	?	?	?	?	?	?
Web frontend	?	?	?	?	?	?	?
Background workers	?	?	?	?	?	?	?
User DB	?	?	?	?	?	?	?
Analytics DB	?	?	?	?	?	?	?
Queue	?	?	?	?	?	?	?



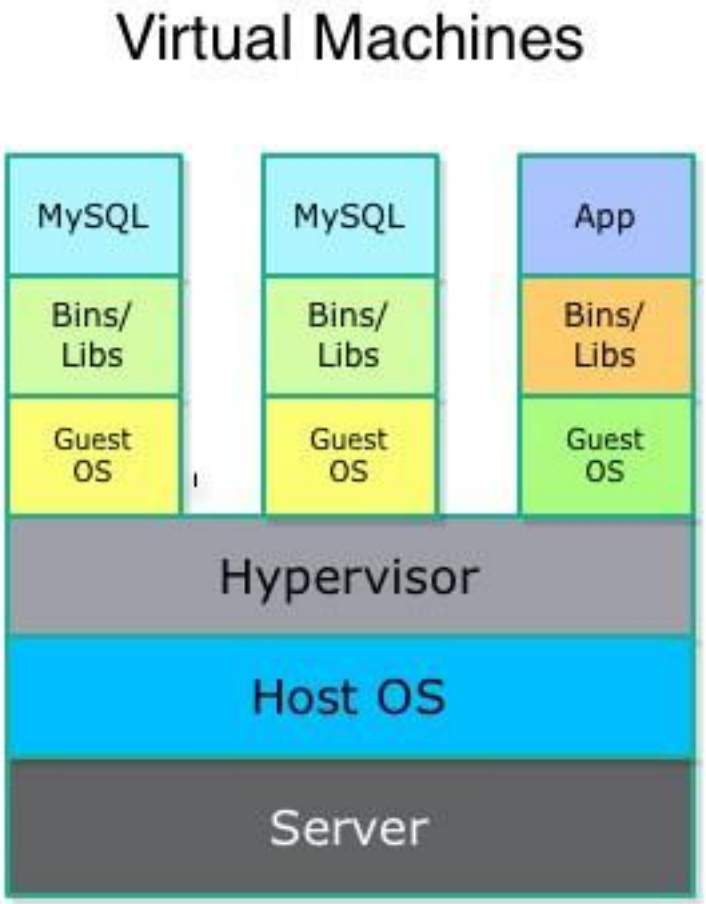
Container for Software

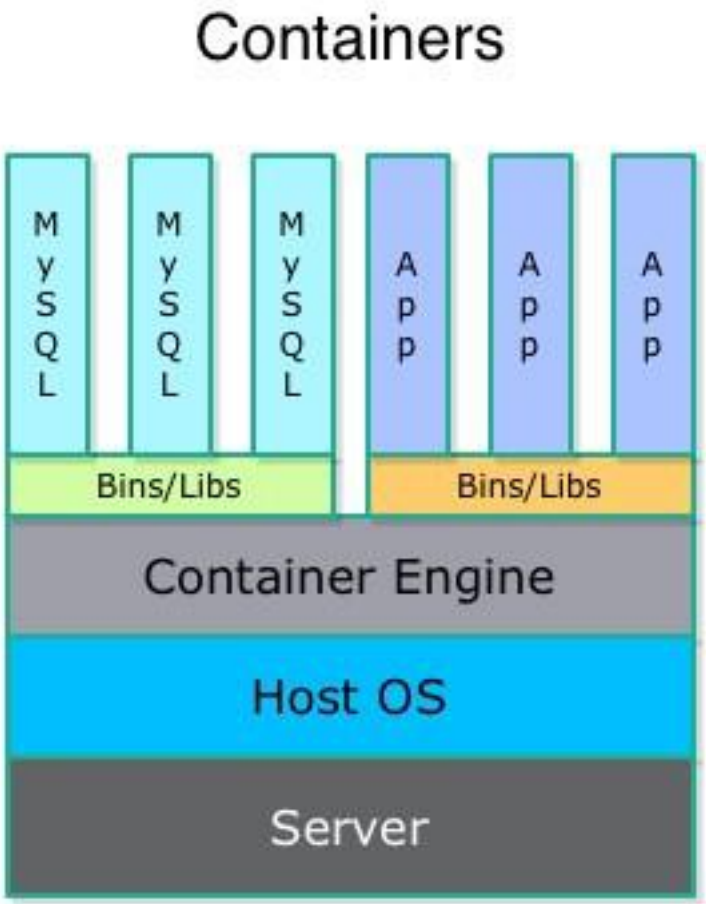


Docker

- Developed by Solomon Hykes
- Initial release on March 2013
- Provide simple operating-system-level virtualization
- Uses Linux Kernel resource isolation

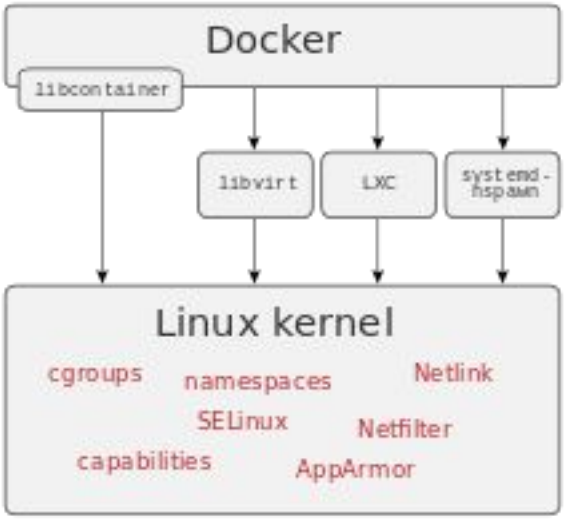
Containers vs. Virtual Machines





Docker on Linux

Docker uses the resource isolation features of the Linux kernel such as cgroups and kernel namespaces, to allow independent containers to run within a single Linux instance, avoiding the overhead of starting and maintaining virtual machines.



Docker Daemon

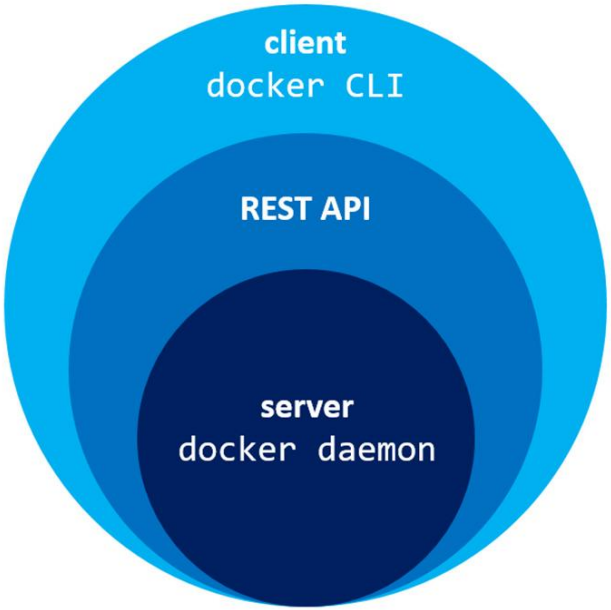
The Docker daemon is a service that runs on your host operating system.

It currently only runs on Linux because it depends on a number of Linux kernel features.

Docker runs on MacOS and Windows too, by virtualizing Linux .

The Docker daemon itself exposes a REST API and has a Command Line Interface (CLI).

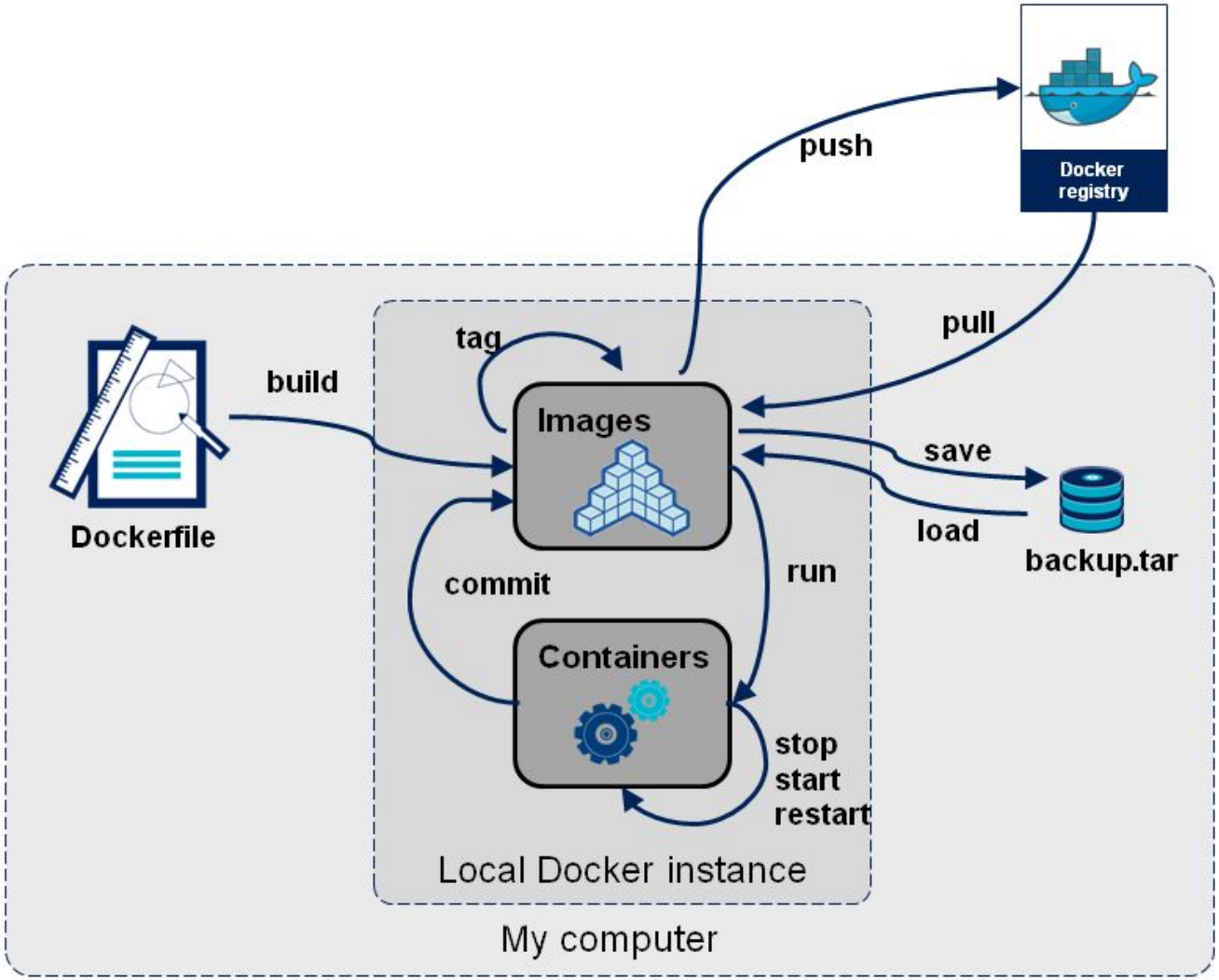
The Daemon must be started.



Container vs Image

A container is an instance of an image.

Images are available from a registry.



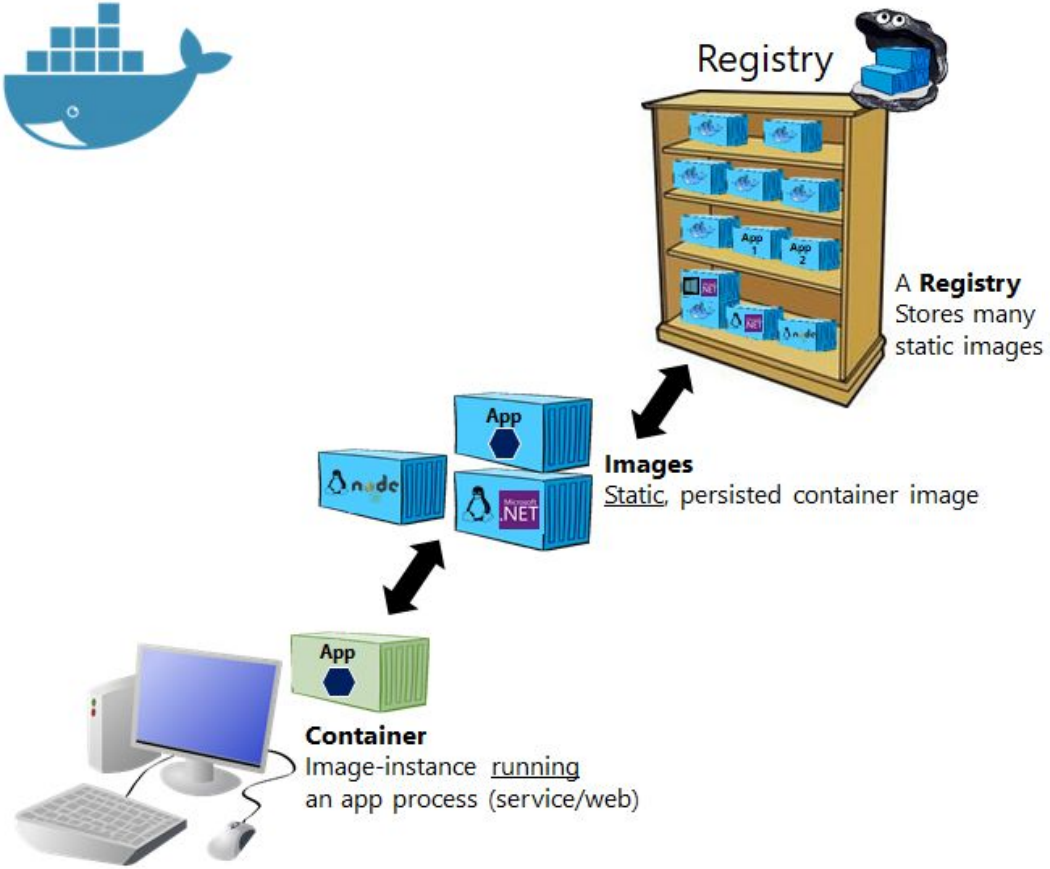
Docker Registry

A Docker Registry stores images.

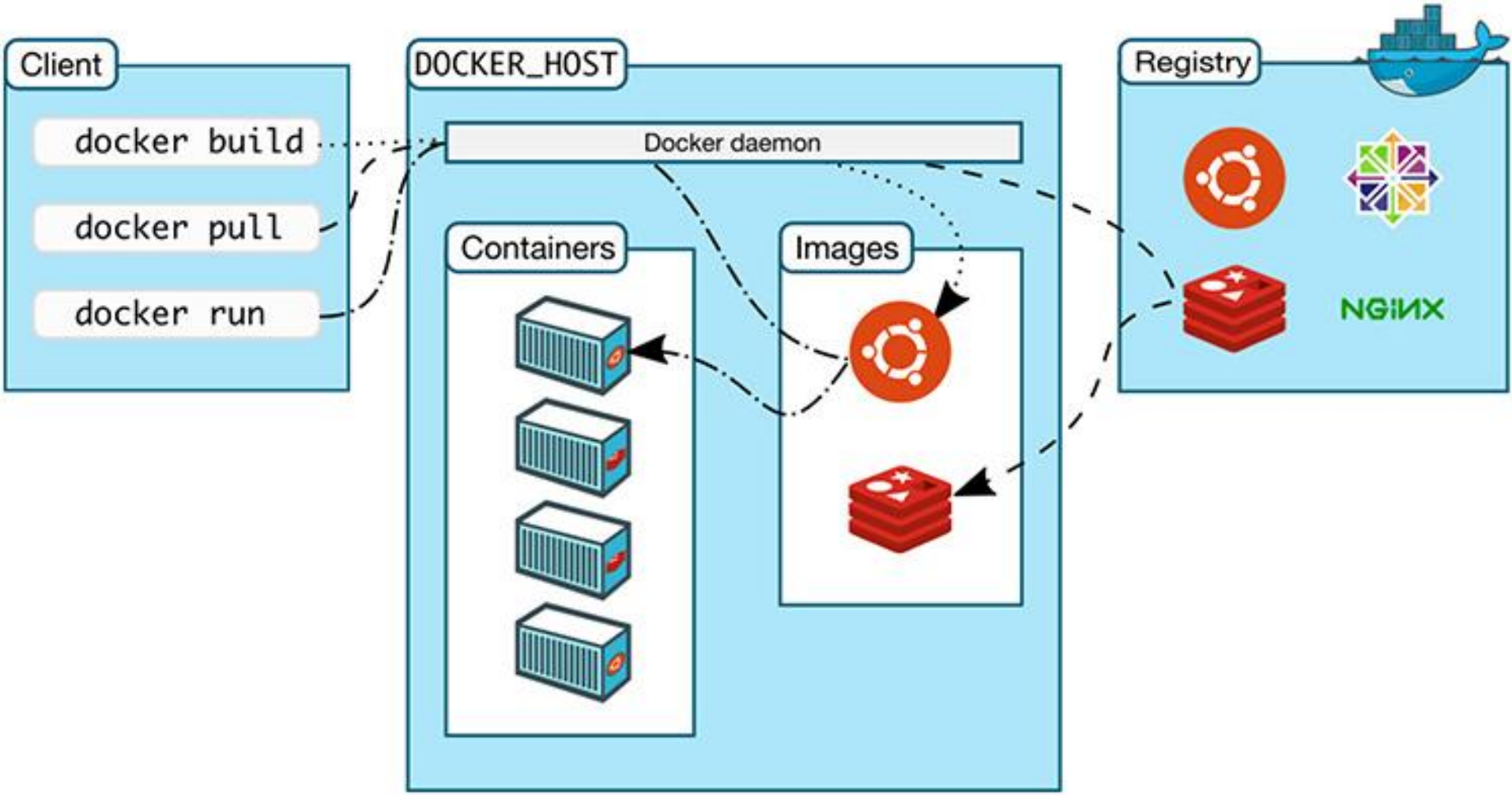
Docker Hub is a public registry with millions of public images.

Popular images ubuntu, mongo, node, openjdk.

Images can be constructed based on other images.



Docker Parts together



Docker CLI

Hochschule Konstanz
Fakultät Informatik

An Intro into Docker CLI: A Cheat Sheet

Docker CLI cheatsheet

docker build

```
docker build [options] .
  -t "app/container_name"    # name
```

Create an image from a Dockerfile.

docker run

```
docker run [options] IMAGE
  # see `docker create` for options
```

Run a command in an image.

Manage containers

docker create

```
docker create [options] IMAGE
  -a, --attach              # attach stdout/err
  -i, --interactive         # attach stdin (interactive)
  -t, --tty                 # pseudo-tty
      --name NAME           # name your image
  -p, --publish 5000:5000   # port map
      --expose 5432         # expose a port to linked containers
  -P, --publish-all         # publish all ports
```

docker exec

```
docker exec [options] CONTAINER COMMAND
  -d, --detach        # run in background
  -i, --interactive   # stdin
  -t, --tty           # interactive
```

Example

```
$ docker exec app_web_1 tail logs/development.log
```

Docker commands

```
docker build -t friendlyname .  # Create image using this directory's Dockerfile
docker run -p 4000:80 friendlyname  # Run "friendlyname" mapping port 4000 to 80
docker run -d -p 4000:80 friendlyname         # Same thing, but in detached mode
docker ps                                  # See a list of all running containers
docker stop <hash>                       # Gracefully stop the specified container
docker ps -a           # See a list of all containers, even the ones not running
docker kill <hash>                     # Force shutdown of the specified container
docker rm <hash>              # Remove the specified container from this machine
docker rm $(docker ps -a -q)             # Remove all containers from this machine
docker images -a                                # Show all images on this machine
docker rmi <imagename>            # Remove the specified image from this machine
docker rmi $(docker images -q)             # Remove all images from this machine
docker login             # Log in this CLI session using your Docker credentials
docker tag <image> username/repository:tag  # Tag <image> for upload to registry
docker push username/repository:tag            # Upload tagged image to registry
docker run username/repository:tag                  # Run image from a registry
```

Docker Hello World

Pull an image of ubuntu, start it, and print Hello World

- `docker run ubuntu echo Hello World`
  - `docker images [-a]`
  - `docker ps -a`

Dockerfile

Dockerfiles are used to

- build images
- modify downloaded images
- define what runs in the instance of the image - the container

Commands in the Dockerfile are Capitalized

Dockerfile commands

FROM: container is build from a base image

COPY: Add files to the container

CMD: specifies Linux commands to run on the container

RUN:

Example Dockerfile

```
#
# Scala and sbt Dockerfile
#
# https://github.com/hseeberger/scala-sbt
#

# Pull base image
FROM  openjdk:8

ENV SCALA_VERSION 2.12.1
ENV SBT_VERSION 0.13.15

# Scala expects this file
RUN touch /usr/lib/jvm/java-8-openjdk-amd64/release

# Install Scala
## Piping curl directly in tar
RUN \
  curl -fsL http://downloads.typesafe.com/scala/$SCALA_VERSION/scala-$SCALA_VERSION.tgz | tar
  echo >> /root/.bashrc && \
  echo 'export PATH=~/scala-$SCALA_VERSION/bin:$PATH' >> /root/.bashrc

# Install sbt
RUN \
  curl -L -o sbt-$SBT_VERSION.deb http://dl.bintray.com/sbt/debian/sbt-$SBT_VERSION.deb && \
  dpkg -i sbt-$SBT_VERSION.deb && \
  rm sbt-$SBT_VERSION.deb && \
  apt-get update && \
  apt-get install sbt && \
  sbt sbtVersion

# Define working directory
WORKDIR /root
```

Output of docker build

```
philipp:~/example$ docker build -t example .
Sending build context to Docker daemon 3.072 kB
Step 1 : FROM centos
latest: Pulling from library/centos
a3ed95caeb02: Pull complete
1534505fcbc6: Pull complete
Digest: sha256:f1f827eca5be4b91ebf0916fa69d1f41234b84752a04360f034cad25c9e1138d
Status: Downloaded newer image for centos:latest
 ---> 96eecaf1019a
Step 2 : MAINTAINER "Philipp Daniels <phdaniel@htwg-konstanz.de>"
 ---> Running in 4c82b6f682f6
 ---> ad82253a0edf
Removing intermediate container 4c82b6f682f6
Step 3 : ENV MY_EXAMPLE "Hello World!"
 ---> Running in 29708a25051d
 ---> 055e76bd91c4
Removing intermediate container 29708a25051d
Step 4 : ADD cmd.sh /
 ---> 5baffc7d62a0
Removing intermediate container 5a74726b8d4d
Step 5 : CMD /cmd.sh
 ---> Running in e1c88a454c30
 ---> a8738a97b1cc
Removing intermediate container e1c88a454c30
Successfully built a8738a97b1cc
```

Docker Container Management

Often Docker Containers need to be jointly operated.

This can be done:

- manually
- with Docker Compose
  - small scale, single server, development oriented, simple
- with Docker Swarm
  - medium scale, on a small cluster
- with Kubernetes
  - large scale, production oriented, complex

Docker Compose

Docker Compose is a tool for defining and running multi-container Docker applications.

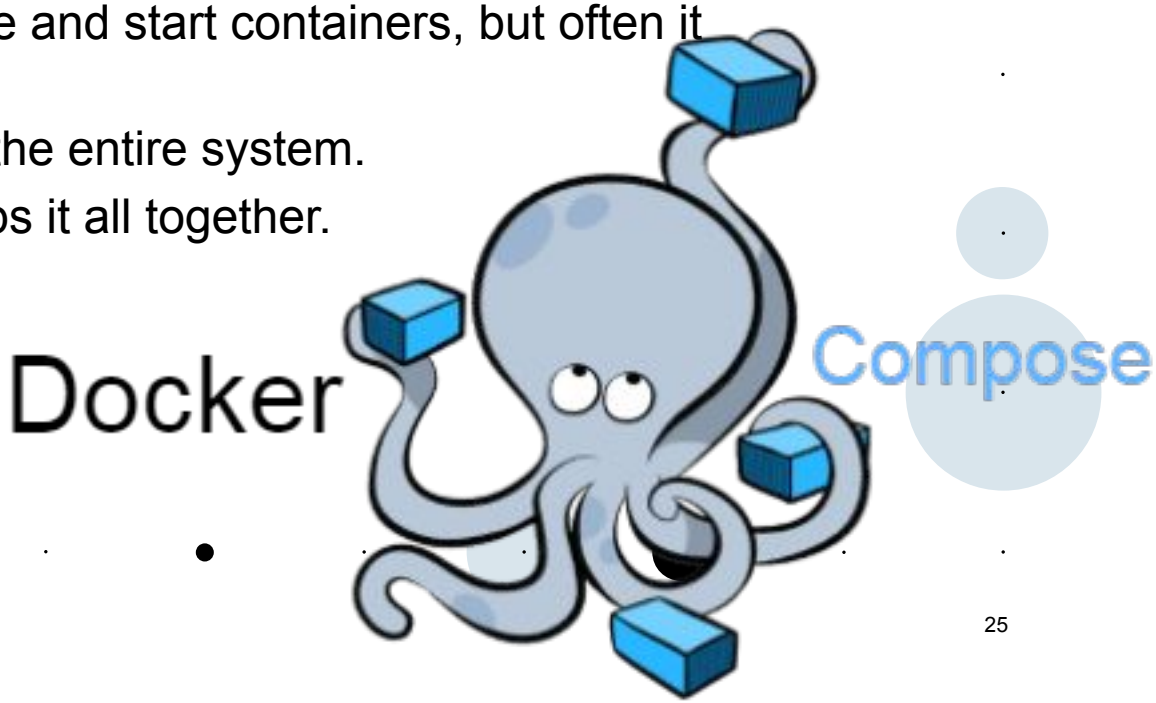
For Microservices it can be used to start all services at once.

Compose is configured in a YAML file that defines application's services.

Compose can also configure and start containers, but often it refers to Dockerfiles.

docker-compose up starts the entire system.

docker-compose down stops it all together.



YAML - YAML Ain’t markup language

YAML is similar to XML or JSON.

- XML is designed to work well for large data and automated processing
- JSON is designed to work well for data exchange, especially to the front end
- YAML is optimised for human-readable configuration files.

YAML is very simple

- Basic types are String, Number, Boolean, Map, Sequence
- Key-value pairs, separated by colon: key: value
- Indentation with 2 spaces creates maps
- - creates sequence
- # is a comment

Example YAML file

```yaml
# Scalar Data Types (strings, numbers, booleans, null)
animal_name: Tiger      # String (no quotes needed for simple strings)
age: 5                  # Integer
weight: 200.5           # Float
is_wild: true           # Boolean
owner: null             # Null value

# String with Special Characters (requires quotes)
habitat: "Savanna, Forest"  # String with comma, quoted to avoid parsing issues

# Multi-line String (using | for literal style)
description: |
  The tiger is a large carnivore,
  known for its distinctive orange coat
  with black stripes.
```

```yaml
# Mapping (Key-Value Pairs)
lion:
  name: Simba           # String
  age: 3                # Integer
  is_leader: true       # Boolean
  territory: Pride Lands
# List (Sequence of Items)
pets:
  - Cat                 # List item 1
  - Dog                 # List item 2
  - Parrot              # List item 3

# Nested Structure (Mapping with List)
zoo:
  name: City Zoo
  animals:
    - species: Elephant
      weight: 6000.0
      diet: Herbivore
    - species: Penguin
      weight: 30.0
        diet: Carnivore
```

Docker Compose Terminology

Services: Define containers, images, ports, and environment variables.

Networks: Enable communication between containers.

Volumes: Persist data across container lifecycles.

Docker Compose file

```yaml
version: '2'
services:
  mongodb:
    image: mongo:3.4.13
    volumes:
      - ./mongodb:/data/db
    ports:
      - 127.0.0.1:27017:27017
  webapp:
    image: node:10.6.0 # Alpine cannot be used, because git executable is needed
    volumes:
      - ./webapp:/src
    command: "bash -c 'yarn install && yarn run dev'"
    working_dir: /src
  api:
    image: zeta-sbt:1.0.6
    build: ./api
    volumes:
      - ./api:/app
      - ./sbt/server:/root
    environment:
      - APPLICATION_SECRET=XXXXX
      - ZETA_MONGODB_SERVER=mongodb
      - ZETA_DEPLOYMENT=production
    stdin_open: true
```

Key Commands

To start all the services defined in your compose.yaml file:

```
$ docker compose up
```

To stop and remove the running services:

```
$ docker compose down
```

To monitor the output of your running containers and debug issues, you can view the logs with:

```
$ docker compose logs
```

To lists all the services along with their current status:

```
$ docker compose ps
```

Limitations of Docker Compose

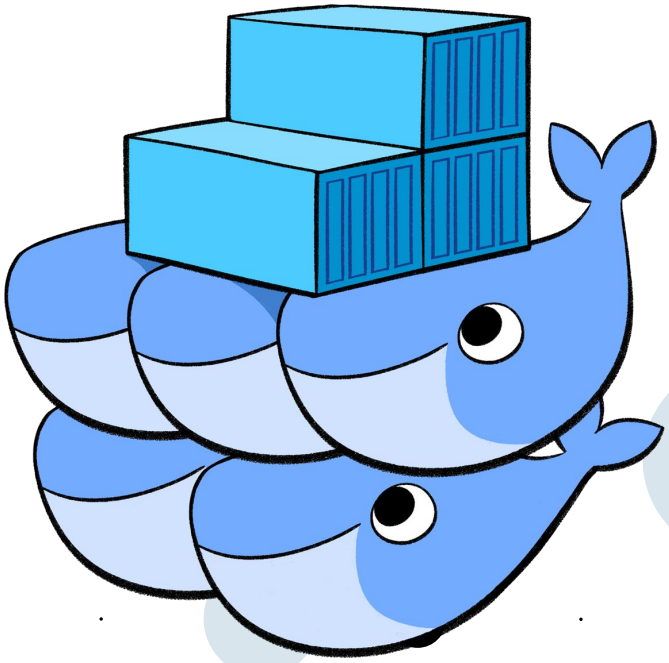
Not designed for large-scale production orchestration.

Lacks

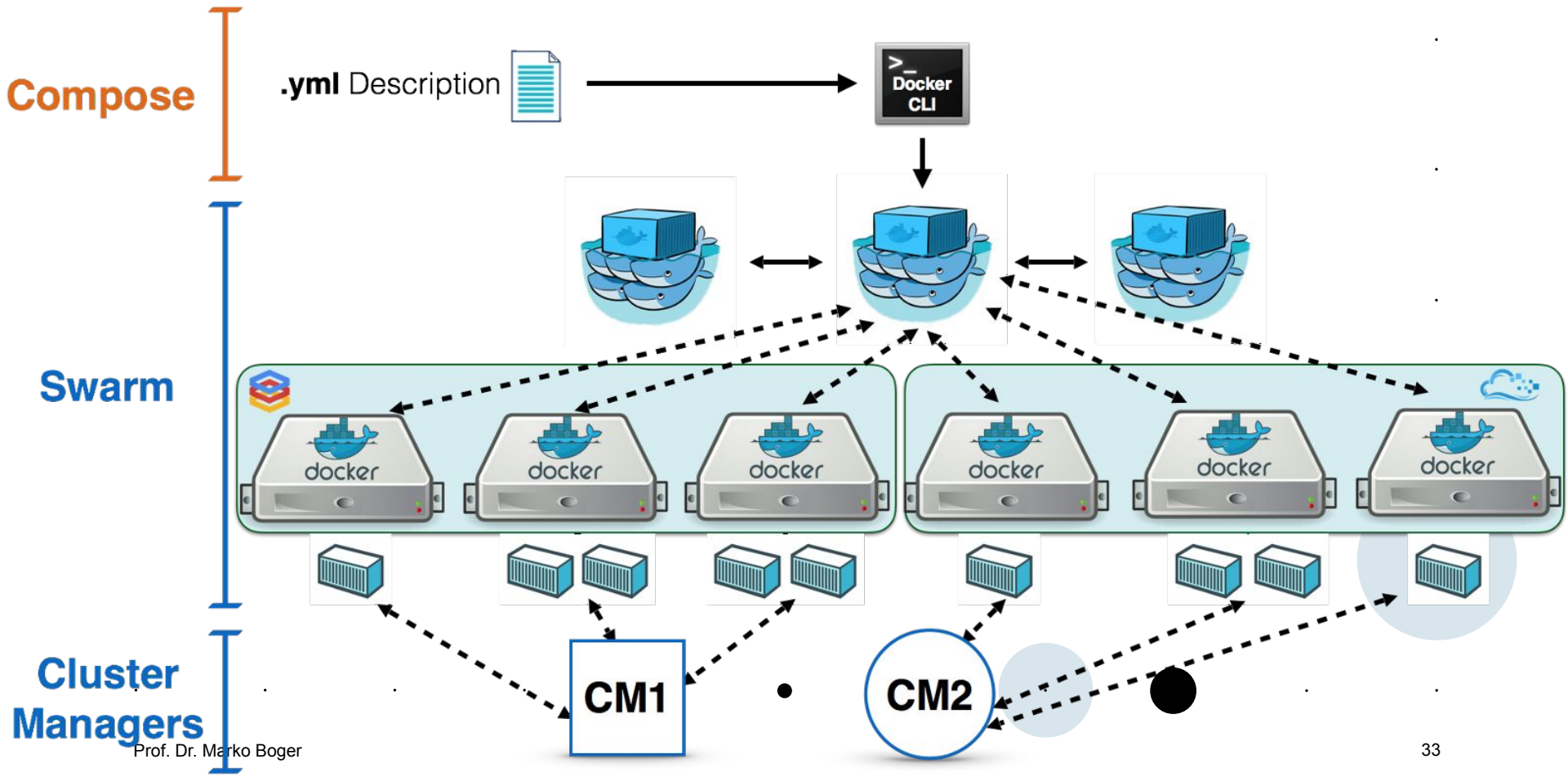
- advanced scheduling
- self-healing
- cluster management

Docker Swarm

Docker Swarm is used to build a cluster on which a docker container system can run.



Docker Swarm



Kubernetes

Kubernetes is an open-source container orchestration platform that automates the deployment, scaling, and management of containerized applications.

Task 5

Start each of your micro services using Docker.
Then start the entire application using Docker-Compose.